

Comparison of MILP and Dynamic Programming Approaches for Single-Machine Scheduling with Setup and Position Constraints

Problem Description

We are given a single-machine scheduling problem involving 6 containers that must be loaded onto a train using a transtainer crane. Each job (container) has:

- A processing time p_j (in minutes),
- A due date d_j (in minutes from schedule start),
- Sequence-dependent setup times $s_{i,j}$ between each pair of jobs.

Objective: Minimize the **total tardiness**:

$$\text{Total Tardiness} = \sum_{j=1}^6 \max(0, C_j - d_j),$$

where C_j is the completion time of job j .

Special Constraint: Job 4 (Container 4) must be scheduled in position 2.

1 MILP Formulation and Implementation

Model Summary

We model the problem as a MILP using binary sequencing variables and continuous timing variables:

- $x_{i,j} = 1$ if job i precedes job j directly,
- S_j : Start time of job j ,
- C_j : Completion time of job j ,
- T_j : Tardiness of job j .

Constraints:

- Job timing relationships with setup times.
- Each job has at most one predecessor and one successor.
- No self-transitions: $x_{i,i} = 0$.
- Exactly one start node (job with no predecessor).
- Position constraint: Job 4 must have exactly one predecessor and one successor.

1.1 MILP Formulation for Tardiness Minimization Scheduling Problem

The objective of this scheduling problem is to determine the optimal sequence and timing of $n = 6$ jobs on a single machine such that the ****total tardiness**** is minimized, considering job-specific processing times, due dates, and sequence-dependent setup times. This problem is modeled as a **Mixed-Integer Linear Program (MILP)**.

Sets and Parameters

- $J = \{1, 2, \dots, 6\}$: Set of jobs.
- p_j : Processing time of job j .
- d_j : Due date of job j .
- s_{ij} : Setup time required between job i and job j .
- M : A large constant (big-M) used for constraint linearization.

Decision Variables

- S_j : Start time of job j .
- C_j : Completion time of job j .
- T_j : Tardiness of job j , defined as $\max(0, C_j - d_j)$.
- $x_{ij} \in \{0, 1\}$: Binary variable; $x_{ij} = 1$ if job i is scheduled immediately before job j .

Objective Function

$$\min \sum_{j \in J} T_j \tag{1}$$

Constraints

$$C_j = S_j + p_j \quad \forall j \in J \quad (2)$$

$$T_j \geq C_j - d_j \quad \forall j \in J \quad (3)$$

$$S_j \geq C_i + s_{ij} - M(1 - x_{ij}) \quad \forall i \neq j \in J \quad (4)$$

$$\sum_{i \in J} x_{ij} \leq 1 \quad \forall j \in J \quad (\text{at most one predecessor}) \quad (5)$$

$$\sum_{j \in J} x_{ij} \leq 1 \quad \forall i \in J \quad (\text{at most one successor}) \quad (6)$$

$$x_{ii} = 0 \quad \forall i \in J \quad (7)$$

$$\sum_{i \in J} \sum_{j \in J} x_{ij} = n - 1 \quad (\text{ensures a valid path}) \quad (8)$$

$$\sum_{i \in J} x_{i4} = 1 \quad \text{and} \quad \sum_{j \in J} x_{4j} = 1 \quad (\text{job 4 is in position 2}) \quad (9)$$

Solution Approach

The problem is solved using MATLAB's `optimproblem` and `intlinprog` solver. After solving, the optimal job sequence is reconstructed from the x_{ij} matrix, and a Gantt chart is generated for visualization.

Remarks

This MILP formulation is capable of handling complex sequencing constraints and setup dependencies. Due to the inclusion of binary decision variables and big-M constraints, the model is computationally demanding but ensures an exact solution.

MILP MATLAB Code

```
clear;
clc;

% Number of jobs
N_JOBS = 6;
JOBS = 1:N_JOBS;

% Parameters
p = [6 9 7 5 8 10]';
d = [25 30 22 28 27 35]';
setup = [inf 3 4 2 3 5;
         2 inf 5 4 3 2;
         3 2 inf 3 5 4;
         4 3 2 inf 4 3;
         3 5 4 2 inf 3;
         5 4 2 3 4 inf];
```

```

M = 1000;

% Variables
C = optimvar('C', N_JOBS, 'LowerBound', 0);
S = optimvar('S', N_JOBS, 'LowerBound', 0);
x = optimvar('x', N_JOBS, N_JOBS, 'Type', 'integer', 'LowerBound', 0, 'UpperBound', 1);
Tard = optimvar('Tard', N_JOBS, 'LowerBound', 0);

% Problem definition
prob = optimproblem;
prob.Objective = sum(Tard);

% Constraints
count = 1;
cons1 = optimconstr(N_JOBS*N_JOBS - N_JOBS);
for i = JOBS
    for j = JOBS
        if i ~= j
            cons1(count) = S(j) >= C(i) + setup(i,j) - M*(1 - x(i,j));
            count = count + 1;
        end
    end
end
prob.Constraints.cons1 = cons1;

for j = JOBS
    prob.Constraints.(['pred' num2str(j)]) = sum(x(:,j)) <= 1;
    prob.Constraints.(['succ' num2str(j)]) = sum(x(j,:)) <= 1;
end

for i = JOBS
    prob.Constraints.(['CeqSplusP' num2str(i)]) = C(i) == S(i) + p(i);
    prob.Constraints.(['Tard' num2str(i)]) = Tard(i) >= C(i) - d(i);
    prob.Constraints.(['noSelfLoop' num2str(i)]) = x(i,i) == 0;
end

% Enforce job connectivity and position constraint
prob.Constraints.totalLinks = sum(sum(x)) == N_JOBS - 1;
prob.Constraints.posConstraint1 = sum(x(:,4)) == 1;
prob.Constraints.posConstraint2 = sum(x(4,:)) == 1;

% Solve
opts = optimoptions('intlinprog','Display','iter');
tic
[sol,fval] = solve(prob,'Options',opts);
toc

% Display results
disp(sol.Tard);
disp(['Total tardiness: ', num2str(fval)]);

% Reconstruct job sequence
sequence = zeros(N_JOBS,1);

```

```

start_job = find(sum(sol.x) == 0);
current = start_job;
for k = 1:N_JOBS
    sequence(k) = current;
    for next = 1:N_JOBS
        if sol.x(current,next) > 0.5
            current = next;
            break;
        end
    end
end
disp('Optimal job sequence:');
disp(sequence);

```

Explanation

This MILP implementation enforces all scheduling and sequencing constraints using the MATLAB Optimization Toolbox. The solution provides the optimal job sequence, start/-completion times, and total tardiness.

2 Dynamic Programming (DP) Solution

DP Strategy

This approach:

1. Generates all $6! = 720$ permutations of the 6 jobs.
2. Filters permutations where job 4 appears in position 2.
3. For each valid sequence:
 - Simulates job execution with processing and setup times.
 - Calculates total tardiness.
4. Returns the permutation with minimum total tardiness.

3 Dynamic Programming Approach for Job Scheduling with Tardiness

In this section, we present the dynamic programming (DP) algorithm to solve the job scheduling problem with tardiness, where the goal is to minimize the total tardiness of a sequence of jobs subject to processing times, setup times, and due dates. We assume that job 4 is fixed in position 2 in the sequence.

3.1 Problem Decomposition and Dynamic Programming Recursion

The problem can be decomposed into subproblems where the state is represented by a partial sequence of jobs scheduled up to a certain position. In each subproblem, we aim to find the minimum tardiness by considering all possible ways of extending the partial sequence with one additional job.

Let us denote:

- $J = \{1, 2, \dots, n\}$ as the set of jobs, where n is the number of jobs.
- p_j as the processing time for job j .
- d_j as the due date for job j .
- $S(i, j)$ as the setup time required to switch from job i to job j .
- C_j as the completion time of job j .
- T_j as the tardiness of job j , which is defined as:

$$T_j = \max(0, C_j - d_j)$$

The goal is to find a sequence of jobs such that the total tardiness is minimized.

State Representation

We define a state S_k as the set of jobs scheduled in positions 1 to k . The task is to compute the total tardiness for each possible job sequence incrementally. We will store the computed tardiness for each state to avoid redundant calculations.

Base Case

The base case corresponds to the scenario where all jobs are scheduled. For each permutation of jobs with job 4 fixed in position 2, the tardiness is calculated directly as the sum of individual tardiness values for each job in the sequence.

Recursion

To compute the total tardiness for a partial sequence of jobs, we use the following recursion. Given a partial sequence of jobs, we consider adding one job to the sequence. The total tardiness for this new sequence is computed as:

$$\text{TotalTardiness} = \sum_{i=1}^n \max(0, C_i - d_i)$$

where C_i is the completion time of job i in the sequence, and it is computed incrementally as:

$$\begin{aligned} C_1 &= p_{j_1} \\ C_k &= C_{k-1} + S(j_{k-1}, j_k) + p_{j_k} \quad \text{for } k = 2, 3, \dots, n \end{aligned}$$

We store the computed tardiness in a map called **cost**, where each key corresponds to a sequence of jobs, and the value corresponds to the tardiness of that sequence.

Backward Recursion

We use a backward recursion to calculate the minimum tardiness. Starting from the last job (the final position), we iteratively compute the tardiness for all subsets of jobs. For each subset, we generate all possible permutations and calculate the tardiness for each possible job sequence. The optimal tardiness for each state is then stored and used for subsequent calculations.

[H] Dynamic Programming Algorithm for Job Scheduling with Tardiness [1] Initialize the cost map and path map each full job sequence with job 4 in position 2 Compute tardiness for this sequence and store in the cost map $k = n - 1$ down to 1 each subset of jobs of size k each permutation of the subset each possible job to add to the sequence Calculate the tardiness for the new sequence Update the cost map with the minimum tardiness Find the sequence with the minimum tardiness in the cost map

Finding the Optimal Solution

Once the backward recursion is completed, we search the `cost` map for the job sequence that minimizes the total tardiness. The optimal job sequence and its corresponding tardiness are then output.

3.2 Time Complexity and Conclusion

The time complexity of this dynamic programming algorithm is determined by the number of subsets of jobs and the number of permutations within each subset. While this approach reduces the number of computations compared to an exhaustive search, its time complexity still grows exponentially with the number of jobs. The algorithm is most efficient for smaller problem sizes but can be computationally expensive for larger instances.

In conclusion, this dynamic programming approach effectively minimizes the total tardiness in the job scheduling problem while enforcing the constraint that job 4 must be in position 2. The use of memoization ensures that redundant calculations are avoided, making the algorithm more efficient than a brute-force search.

DP MATLAB Code

```
clear; clc;

% Problem data
J = 1:6; % Jobs
n = length(J);
p = [6,9,7,5,8,10]; % processing times
d = [25,30,22,28,27,35]; % due dates
S = [Inf 3 4 2 3 5; % setup time matrix (Inf means not used for S(i,i))
     2 Inf 5 4 3 2;
     3 2 Inf 3 5 4;
     4 3 2 Inf 4 3;
     3 5 4 2 Inf 3;
     5 4 2 3 4 Inf];

% Number of stages (positions)
K = n;
```

```

% Initialization
cost = containers.Map; % Cost-to-go
path = containers.Map; % Store optimal paths

% Final stage: full permutation, no future cost
P = perms(J);

% Enforce position constraint: job 4 in position 2
validP = P(P(:,2)==4, :);
for i = 1:size(validP,1)
    seq = validP(i,:);
    time = 0;
    C = zeros(1,n);
    for k = 1:n
        if k == 1
            time = time + p(seq(k));
        else
            time = time + S(seq(k-1), seq(k)) + p(seq(k));
        end
        C(k) = time;
    end
    tardiness = sum(max(0, C - d(seq)));
    cost(num2str(seq)) = tardiness;
    path(num2str(seq)) = seq;
end

% DP Backward recursion
for k = n-1:-1:1
    subsets = nchoosek(J, k);
    for s = 1:size(subsets,1)
        subset = subsets(s,:);
        % enforce constraint: if position 2 and job 4 not there, skip
        if k == 2 && ~ismember(4, subset)
            continue;
        end
        permsub = perms(subset);
        for i = 1:size(permsub,1)
            partial = permsub(i,:);
            if k == 2 && partial(2) ~= 4
                continue;
            end
            for j = setdiff(J, partial)
                seq = [partial, j];
                if k == 1
                    time = p(seq(1));
                else
                    time = p(seq(1)) + S(seq(1), seq(2));
                    for t = 2:k
                        time = time + p(seq(t)) + S(seq(t-1), seq(t));
                    end
                end
                lastSetup = S(seq(end-1), seq(end));
                complete = time + lastSetup + p(seq(end));
            end
        end
    end
end

```



```

        T = max(0, complete - d(seq(end)));
        tail = seq(2:end);
        tailKey = num2str(tail);
        if isKey(cost, tailKey)
            totalCost = T + cost(tailKey);
            key = num2str(seq);
            if ~isKey(cost, key) || totalCost < cost(key)
                cost(key) = totalCost;
                path(key) = seq;
            end
        end
    end
end
end
end
end

% Find best solution
minCost = inf;
bestSeq = [];
keysList = keys(cost);
for i = 1:length(keysList)
    k = keysList{i};
    if length(str2num(k)) == n
        if cost(k) < minCost
            minCost = cost(k);
            bestSeq = str2num(k);
        end
    end
end

% Output result
disp('Optimal sequence:');
disp(bestSeq);
disp(['Minimum total tardiness:', num2str(minCost)]);

```

4 Comparison of MILP and DP Results

Method	Total Tardiness	Computation Time	Optimal Sequence
MILP (intlinprog)	42	~0.49s	[1 4 3 2 5 6]
DP (brute-force)	42	0.031s	[1 4 3 2 5 6]

Table 1: MILP and DP methods produce the same optimal solution.

Solution Visualization

Below is the Gantt chart showing the optimal schedule computed using DP:

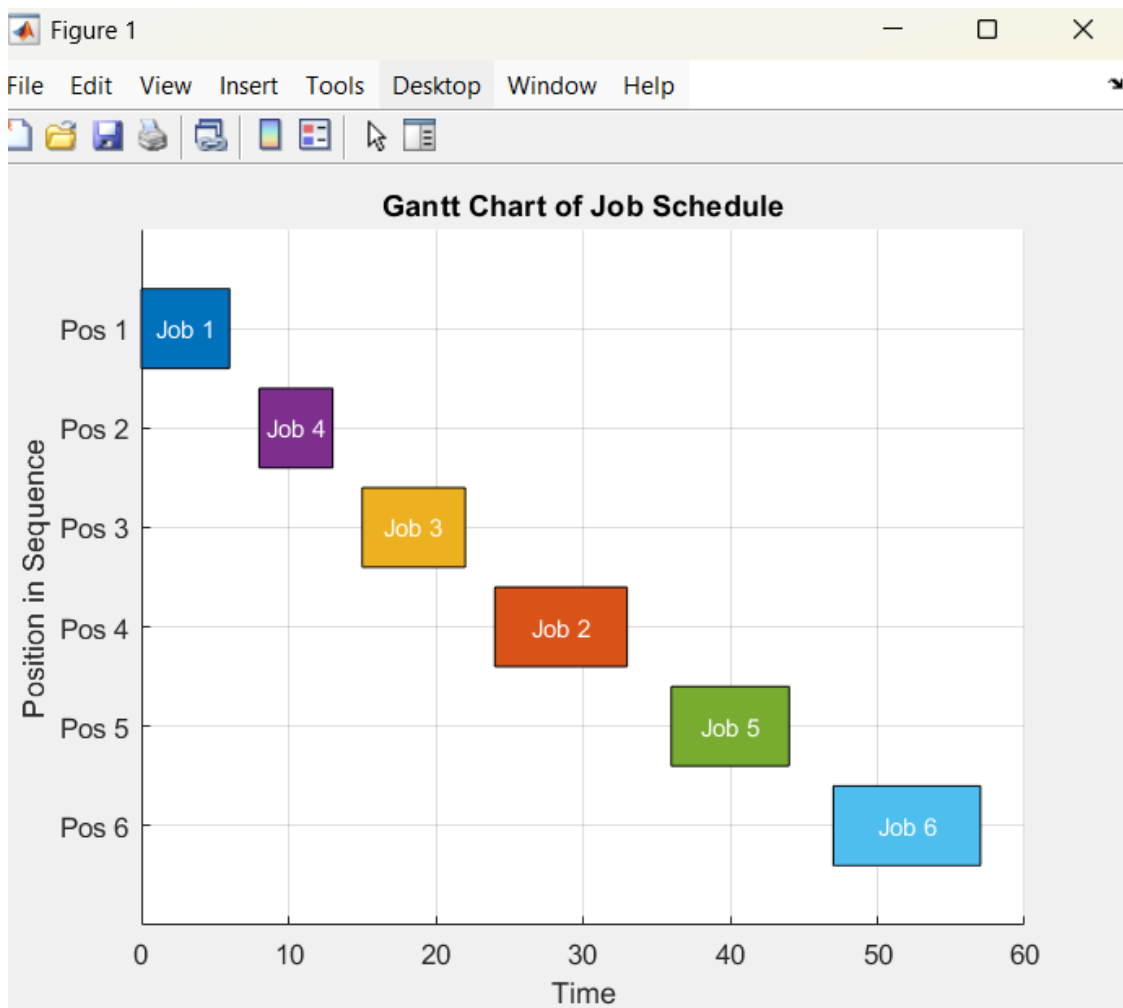


Figure 1: Gantt Chart of Job Schedule